数字人 AI 服务端多机方案

接口 · 状态 · 调度 · 数据同步 · 高可用 — 2026-10-18 方案稿，尚未落地

1. 定位：三层架构，只管 AI 服务端
2. 整体架构与正式环境拓扑
3. 现有状态盘点
4. 高可用：控制面主备
5. 接口：哪里带状态
6. 通信与调度
7. 要不要数据库
8. 对外接口设计（后端 / 前端）
9. 方案复盘：遗漏与优化
10. 目标规模：2 台 8×RTX 4090
11. 落地步骤与验证
12. 离线生产线：闲时视频生成与人设构建

〇、定位：前端 / 业务后端 / AI 服务端 三层，本方案只管 AI 服务端

- 业务后端只对接**一个 AI 服务入口**（控制面的 VIP 地址），不知道有几台 GPU 机。
- 控制面**只做 AI 资源的调度和路由**（选卡、会话路由、节点健康、素材分发），**不做业务逻辑**（用户、权限、计费、人设管理、聊天记录归后端）。
- 控制面是一个进程角色，不独占机器：两台时放在其中一台上，那台同时也是渲染/合成节点，两台一起干活。
- 调用链：前端 → 后端（鉴权）→ 控制面 `POST /v1/sessions` 拿 `session_id + ticket` → 前端凭 ticket 直接和控制面做 WebRTC 信令 / ASR WS → 媒体 UDP 直连所在 GPU 节点。**音视频流不经过后端，也不经过控制面。**
- 数据库：业务数据在后端已有的库里；AI 服务端不需要库，只存按 ID 寻址的素材文件 + 内存运行态。

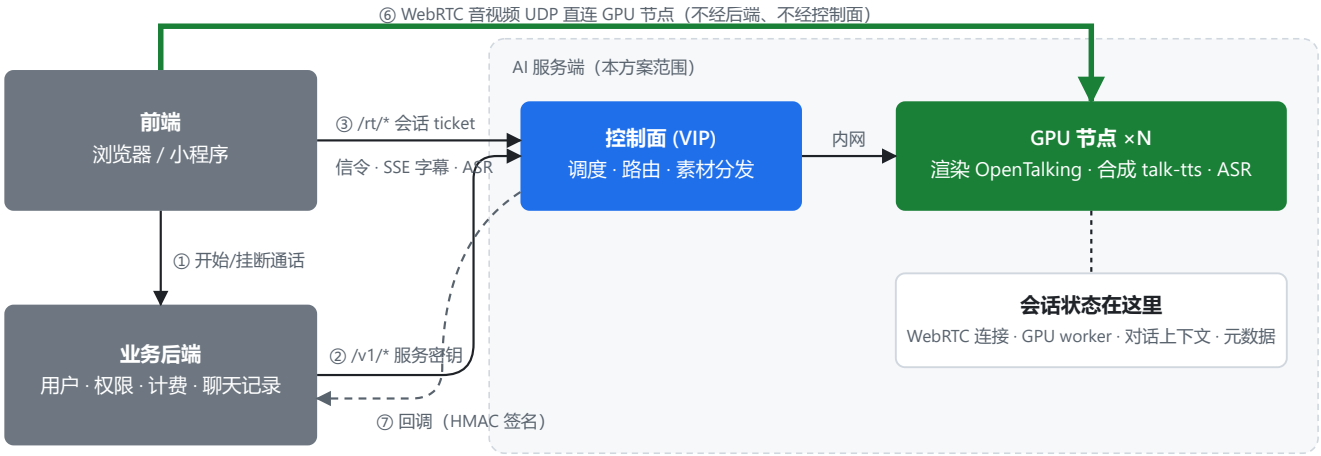


图 1 · 三层关系：后端只认一个 AI 入口；媒体流走直连

一、整体架构与正式环境拓扑

一个控制面 + N 个渲染节点 + M 个合成节点。正式环境从 **2 台 8×RTX 4090** 起步，两台完全对称：每台 = `talk-control`（一主一备，无 GPU）+ `talk-node`（节点网关 + 7 个 OpenTalking 实例，每卡一个）+ `talk-tts`（1 卡）+ coturn。

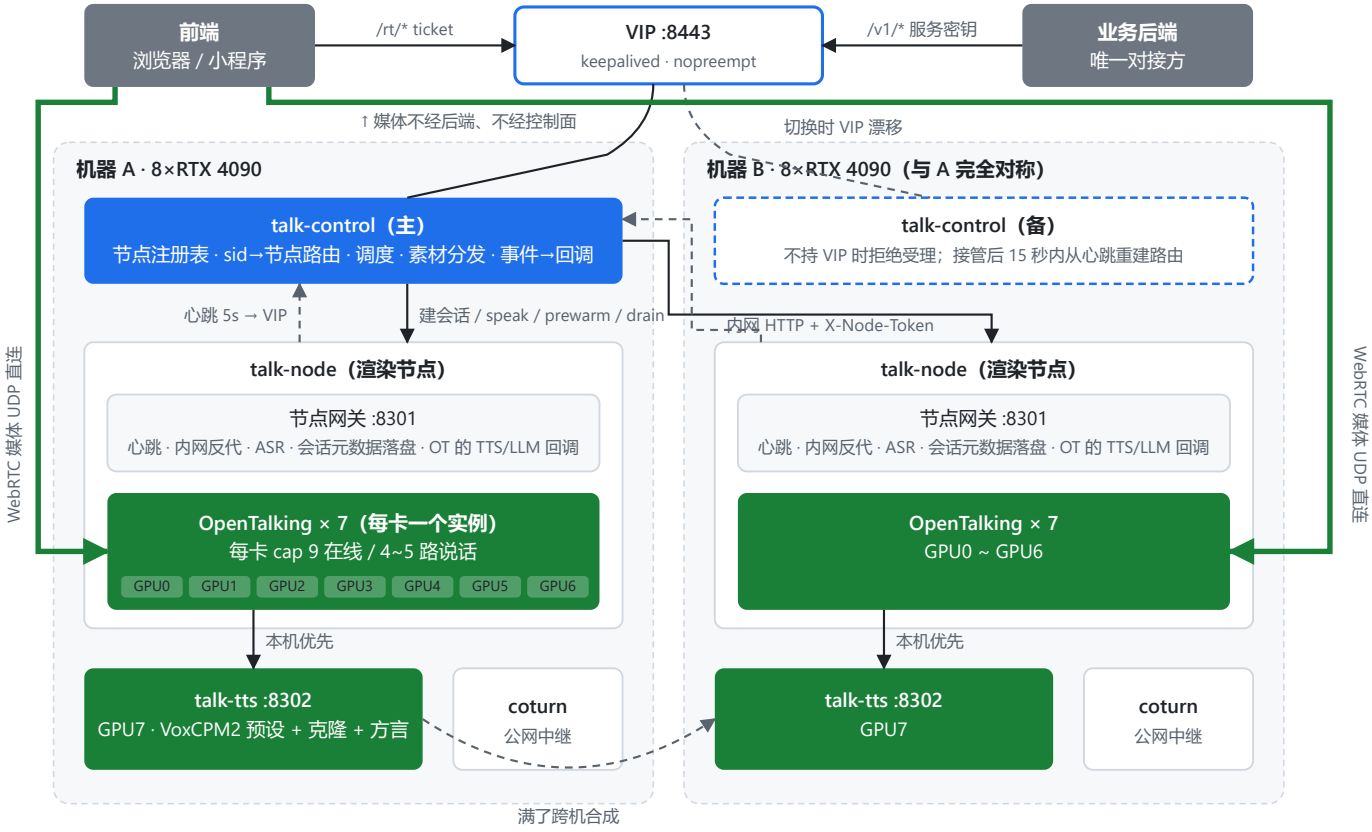


图 2 · 正式环境拓扑：两台 8×4090 对称部署，控制面一主一备

- **控制面**：建会话调度、按 `session_id` 路由、节点健康、素材分发、事件落盘后回调后端。可无 GPU，单独一个容器，重启渲染节点不会把入口一起带掉。
- **渲染节点**：节点网关 + 本机所有 OpenTalking 实例。节点上的网关继续给本机 OT 提供 TTS/LLM 回调（`127.0.0.1:8301`，OT 无需改），并把 TTS 请求转给合成池。ASR 放在节点（随会话走，和机器数一起扩）。
- **合成节点**：talk-tts 无会话状态，任何节点都能合成任何音色，只要本地有参考 wav。
- 扩到第 3、4 台时新机器只跑 `talk-node` + `talk-tts`，控制面仍是前两台主备。

二、现有状态盘点（决定了多机要解决什么）

状态	现在在哪	多机后问题
会话→实例 `_owner`、预热表 `_warm`	网关内存（`gateway/ot_pool.py`）	只有一个入口就没问题；多入口要共享
会话本身（WebRTC 连接、GPU worker、对话历史）	OpenTalking 进程内存	**天然绑在某台机某张卡**，无法迁移
建形象任务进度 `JOBS`	网关内存（`animate.py`）	重启即丢
LLM 选择	`state/gateway.json`	小配置，单写者即可
克隆音色登记 + 参考 wav	`state/voices.json`、`state/voices/*.wav`	**合成节点都要能读到**
形象素材 + 人脸缓存	`/w/avatars`	**所有渲染节点都要有**
模型权重	每机 `/w/models`	部署时分发，不需同步
OT→网关回调	写死 `127.0.0.1:8301`，只许本机	每个节点保留本地 8301 即可，不跨机
WebRTC 媒体	UDP 直连 OT 所在机器（host 网络）	浏览器必须能直连**每台**渲染机

三、高可用：控制面主备

- 备机常驻一个控制面进程，平时不接流量；GPU 照常干活。只有「控制面」这个角色是主备。
- 切换：同网段用 keepalived 漂一个 VIP 给后端，主机挂了 VIP 自动漂到备机，后端无感；设 `nopreempt`，主机恢复后不抢回。keepalived 加 `vrrp_script` 检查控制面进程健康，不只看机器通不通。
- 运行态不用同步：会话的完整元数据（含 callback_url、biz_ref、llm 配置）存在**会话所在节点**，心跳带给控制面；备机接管后 15 秒内重建路由表。ticket 是 HMAC 签名的无状态令牌，切换后仍有效。
- 脑裂防护：备机不持有 VIP 时拒绝 `/v1`、`/rt`（503 + Retry-After）；节点心跳只发 VIP。
- 主机上的会话随机器一起断（GPU 在那台上），前端重连 → 落到备机；备机上的会话不受影响。
- 必须去掉的单点：素材每台节点本地存全量（不用单机 NFS）；证书两台同一份；TURN 两台各一个。
- 降级：只剩一台时容量减半，满了返回 503，后端提示排队。

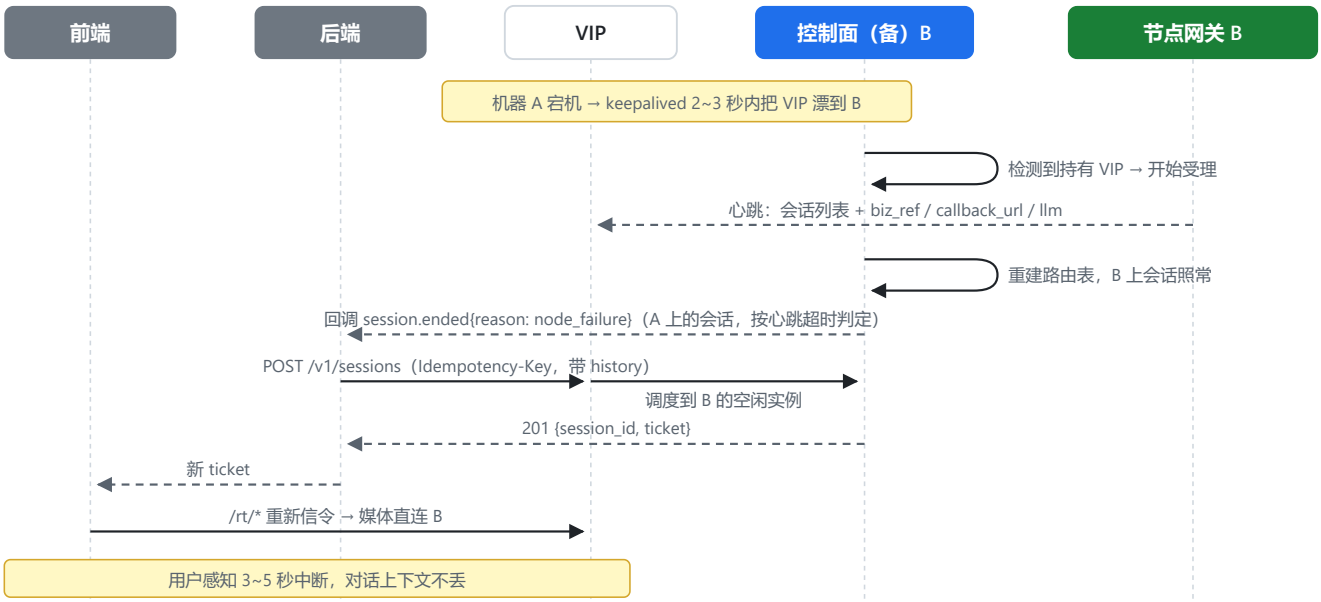


图 3 · 主机整机宕机时的切换时序

四、接口：哪里带状态

> **原则**
> 会话有状态且粘在一张卡上；其余一律无状态、ID 全局可解析。

1. **对外只暴露控制面的 `/v1/*`（后端）和 `/rt/*`（前端）。** `session_id` 是有状态句柄，所有会话请求都带它，控制面查 `sid → (节点, 实例)` 转发。前端不知道节点地址（除 WebRTC 的 ICE 候选）。
2. **形象 ID、音色 ID 是全局 ID**，任何节点拿到都能解析 → 每台节点都要有全量素材。
3. **内部接口**（控制面 ↔ 节点，内网 + `X-Node-Token`）：
   - 节点 → 控制面 `POST /internal/nodes/heartbeat`（每 5 秒，会话开始/结束时立即补一条增量）：`{node_id, role, base_url, public_ip, gpu_model, instances:[{id, cap, sessions:[…], talking, warm_avatars}], tts:{inflight, max}, assets_hash}`。15 秒没收到判下线。
   - 节点 → 控制面 `POST /internal/events`：对话轮次、会话结束 + 用量；节点本地 spool，失败重试。
   - 控制面 → 节点：沿用 OT 现有 API（`/sessions`、`/avatars/{id}/prewarm`、`/sessions/{id}/speak` …），经节点网关反代加 token。
   - 控制面 → 节点 `/internal/avatars/{id}/sync`、`/internal/voices/sync`、`/internal/drain`。
4. **合成池**：`/v1/audio/speech`、`/voices/register` 已有；渲染节点本机优先，满了按 inflight 最少跨机；合成节点遇到未知音色时向控制面拉参考 wav。

五、通信与调度

- **通信**：控制类全用 HTTP/JSON；ASR 与 OT 事件 WS/SSE 经控制面反代（现有 `ws_proxy` / `http_proxy`）；媒体 WebRTC 直连。这个规模**不需要消息队列。**
- **调度**（把 `ot_pool.pick()` 推广到全局）：候选 = 所有健康、非 draining 节点上的实例；排序 key `（在线数/cap，未预热该形象，节点当前说话数，节点id)`；cap 由节点按显卡上报；先占位再建会话；全满回 503。合成池独立调度。
- **故障**：节点挂了，其上会话直接失效（不做会话迁移），前端重连换节点，后端带 `history`。
- **WebRTC 跨机**：每台渲染机通告自己浏览器可达的 IP；公网部署 coturn 每台一个，`ice-config` 下发会话所在机的 TURN。HTTPS 证书只在控制面。

六、要不要数据库

按数据性质分开处理，**第一阶段不需要数据库**：

数据	方案
运行时态（会话表、负载、预热）	真源在节点，控制面内存缓存，心跳重建；**多个控制面 active-active 时**才上 Redis
会话元数据（callback_url、biz_ref、llm 配置）	存会话所在节点（落盘），心跳带回
小元数据（形象列表、音色登记、人设）	只有控制面写 → JSON/SQLite；节点清单哈希可对账重建
大文件（形象素材、底板视频、克隆 wav）	每台节点本地全量：新建时主控制面推送，节点启动时对账补齐；4 台以上再换 MinIO + 本地缓存
人脸缓存（prewarm 产物）	各节点本地生成，不同步
建形象任务	控制面落盘（`state/jobs/`），重启续跑
模型权重	部署时按角色分发

> **一句话原则**
> 状态跟着 GPU 走，控制面只是可重建的缓存，后端是业务真源。守住这条，主备、扩容、重启都不用做数据同步。

七、对外接口设计（AI 服务端 → 业务后端 / 前端）

通用约定

- 地址 `https://<VIP>:8443`。`/v1/*` 给**业务后端**（服务到服务，`Authorization: Bearer <服务密钥>`）；`/rt/*` 给**前端**（会话 ticket：`X-Session-Ticket` 头，SSE/WS 用 `?ticket=`）。ticket 绑定单个会话，HMAC 签名，短时有效。
- JSON；错误统一 `{"error":{"code","message","retry_after?"}}`。
- 创建类请求支持 `Idempotency-Key`；建会话另以 `biz_ref` 在节点侧兜底去重。
- ID 前缀：`av_` 形象、`vc_` 音色、`sess_` 会话、`job_` 任务。AI 侧不存用户信息，后端传 `biz_ref` 透传。
- 耗时操作（建形象 1~3 分钟）走「202 + job + 回调」。

1. 状态与容量

方法	路径	说明
GET	`/v1/health`	`{status, role: primary|standby, nodes:[{id, ok, instances}]}`
GET	`/v1/capacity`	`{online, max_online, talking, available}`，后端可在「开始通话」前预判是否排队

2. 形象

方法	路径	说明
POST	`/v1/avatars`	multipart：`image`、`name`、`animate`（是否生成动态底板）、`callback_url?`、`biz_ref?` → `202 {avatar_id, job_id}`
GET	`/v1/avatars/{id}`	`{id, name, status: processing|ready|failed, animated, preview_url, created_at}`
GET	`/v1/avatars?type=builtin`	内置形象列表
DELETE	`/v1/avatars/{id}`	从所有节点删除
GET	`/v1/avatars/{id}/preview`	封面图

3. 音色

方法	路径	说明
GET	`/v1/voices/presets`	系统音色分组（含推荐年龄段）+ 支持的方言列表
POST	`/v1/voices/clone`	multipart：`audio`（≤20MB，截前 20 秒）、`name`、`biz_ref?` → `{voice_id, ref_text, status}`
GET	`/v1/voices/{id}`	音色信息
DELETE	`/v1/voices/{id}`	删除（所有合成节点）
GET	`/v1/voices/{id}/preview?dialect=`	试听 mp3

4. 任务

方法	路径	说明
GET	`/v1/jobs/{id}`	`{status: queued|running|done|failed, step, progress, result, error}`

5. 会话（核心）

`POST /v1/sessions`

```
{
  "avatar_id": "av_xxx",
  "voice_id": "vc_xxx", "dialect": "四川话",
  "persona": { "system_prompt": "……" },
  "llm": { "mode": "builtin", "provider": "ark" },
  "history": [{"role":"user","content":"…"},{"role":"assistant","content":"…"}],
  "idle_timeout_s": 120, "max_duration_s": 1800,
  "queue": false, "priority": "normal",
  "biz_ref": "order_123", "callback_url": "https://backend/ai/callback"
}
```

- `llm.mode`：`builtin` AI 侧用内置 LLM（豆包/MiniMax）+ system_prompt；`external` AI 侧调后端提供的 OpenAI 兼容地址（`llm.url`、`llm.api_key`），后端可加知识库、审核、存历史；`none` 不走 LLM，只念后端下发的文本。
- `history`：断线重连/切换节点后接着聊，由后端带回最近几轮。
- 返回 `201 { session_id, ticket, ticket_expires_at, rt_base_url, expires_at }`。
- 满员：`503 capacity_full` + `retry_after`；`queue:true` 时 `202 {state:"queued", position}`，排到后 SSE / 回调通知。

方法	路径	说明
GET	`/v1/sessions/{id}`	`{state: queued|created|live|ended|failed, speaking, started_at, ended_reason?}`
PATCH	`/v1/sessions/{id}`	通话中换 `voice_id` / `dialect` / `system_prompt`
POST	`/v1/sessions/{id}/speak`	`{text, mode: chat|say}`：`chat` 当作用户说的话走 LLM；`say` 原样念（播报、开场白）
POST	`/v1/sessions/{id}/interrupt`	打断当前说话
POST	`/v1/sessions/{id}/ticket`	重新签发 ticket（页面刷新、重连）
DELETE	`/v1/sessions/{id}`	挂断，返回用量 `{duration_s, talk_s, tts_chars, llm_tokens}`

6. 前端实时接口（`/rt/*`，ticket 鉴权）

方法	路径	说明
GET	`/rt/sessions/{id}/ice-config`	STUN/TURN 配置（会话所在机的 TURN）
POST	`/rt/sessions/{id}/webrtc/offer`	`{sdp, type}` → answer（信令经控制面，媒体直连 GPU 节点）
POST	`/rt/sessions/{id}/start`	开始推流
GET	`/rt/sessions/{id}/events`	SSE：`session.state_changed`、`speech.started`、`subtitle.chunk{text, is_final}`、`speech.ended`、`asr.result{text}`、`session.ended{reason}`
WS	`/rt/sessions/{id}/asr`	语音输入：先发 `{"type":"meta"}`，再发 16k s16le PCM，最后 `{"type":"end"}`，回 `{text}` 或 `{error}`
POST	`/rt/sessions/{id}/interrupt`	用户开口即打断
POST	`/rt/sessions/{id}/text`	打字输入 = speak(mode=chat)

前端不能直连 AI 域名时，后端可原样反代 `/rt/*`（HTTP/SSE/WS），但 WebRTC 媒体仍须直连 GPU 节点或 TURN。

7. 回调（AI → 后端，`callback_url`）

- 带 `X-Signature: HMAC-SHA256(body, 回调密钥)`、`event_id`（后端按它去重），失败按 1s/5s/30s/2min 重试；节点侧磁盘 spool 保证不丢。
- 事件：`avatar.ready` / `avatar.failed`、`voice.ready`、`session.started`、`conversation.turn{user_text, bot_text, ts}`（后端以此存聊天记录）、`session.ended{reason: hangup|idle_timeout|max_duration|node_failure|error, usage}`。
- `node_failure`：后端带 `history` 重新建会话，前端用新 ticket 重连。

8. 一次通话的时序

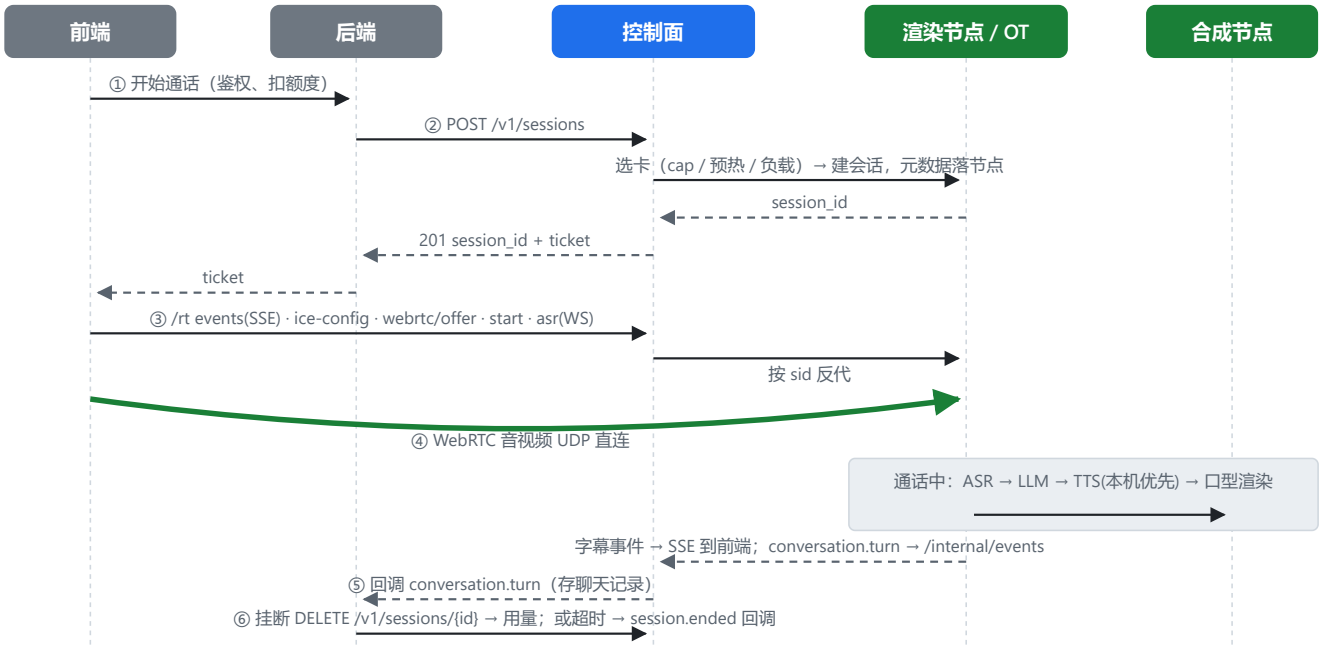


图 4 · 一次通话：控制类走控制面，媒体直连，记录回调给后端

八、方案复盘：遗漏与优化

8.1 必须补上的（不补会出事故）

#	问题	后果	对策
1	**会话元数据只在控制面内存**	主备切换后不知道回调谁、external LLM 调哪、ticket 全失效	**状态放在 GPU 所在节点**，心跳带回，控制面只是缓存；ticket 改 HMAC 无状态令牌
2	**脑裂**：网络分区时两边都以为自己是主	同一张卡被超卖、重复回调	不持 VIP 拒绝受理；心跳只发 VIP；keepalived 加进程健康脚本
3	Idempotency-Key 存内存	切换时重试重复建会话	`biz_ref` 相同且在线的会话直接返回已有 session_id
4	回调送不出去时丢事件	聊天记录缺轮次、`session.ended` 丢了无法计费	节点磁盘 spool + 重试；用量由节点算
5	per-session LLM / 人设配置没下发到节点	`external` 模式做不了	节点网关按 session_id 查本地会话表选 provider / url / key
6	素材推送时节点离线、删除没传到	节点缺形象建会话失败	心跳带清单哈希，控制面对账补推/补删；节点启动先对账
7	建形象任务只在内存	控制面重启任务丢	任务落盘续跑，失败回调 `avatar.failed`
8	idle / max 超时谁执行	控制面挂了会话占着卡	节点本地定时器
9	没有可观测性	出问题只能翻日志	每节点 Prometheus 指标（在线/说话、GPU、首包延迟、TTS RTF、掉帧）；`X-Request-Id` 透传
10	证书 & 时间	切换后 HTTPS 失败、ticket 过期误判	证书两台同份；NTP
11	公网 WebRTC	UDP 过不去就黑屏	固定 UDP 端口段 + PUBLIC_HOST；coturn 每台一个；带宽按 1.5 Mbps/路预留
12	安全细节	`/rt` 被刷、恶意上传、内网接口裸奔	ticket 限速；上传校验 MIME/像素/人脸；`/internal` 只绑内网 + token；密钥双份轮换

8.2 值得优化的

- **本机优先的合成调度**：先用同机 talk-tts，满了再跨机，省一次 LAN 往返（克隆音色首包更敏感）。
- **自适应 cap**：节点上报掉帧率 / GPU 利用率，连续掉帧时临时下调 cap（1:2 超卖是静态估计）。
- **心跳 + 即时增量**：会话开始/结束时立即推一条，避免并发建会话按旧负载挤到同一张卡。

- **热门形象常驻预热池**：所有节点保持 Top-N 形象 warm；新节点上线先预热再接流量。
- **预建会话**：后端在用户进入准备页时就建会话，点通话时直接信令，省 2~3 秒。
- **优先级与预留**：`priority: high` 每节点预留 1 个说话槽。
- **会话录制（可选）**：`record: true` 时节点录音频 + 字幕，结束后上传对象存储并在回调里给 URL。
- **4 台以上再做**：Redis（多控制面 active-active）、MinIO、节点自动注册。
- **不需要做**：消息队列、服务网格、会话迁移、多 Region。

九、目标规模：2 台 8×RTX 4090 起步

> **注意**
> 205（2×3090）/ 206（8×TITAN V）只用来验证架构（拆容器、调度、主备切换）；下面的容量是 4090 口径的**估算**，上线前必须在第一台 4090 上用 `scripts/loadtest.py` 实测。

9.1 和测试环境的差别

- 两台机器完全对称，主备就是镜像关系；没有异构，「按机器快慢加权」「Volta fp16 补丁」第一期都不需要。
- 4090 是 Ada 架构（bf16、flash-attn 都有），直接用 205 的镜像和 `.env`。
- 第 3、4 台只跑 `talk-node` + `talk-tts`，控制面仍是前两台主备。

9.2 单机分工与容量

项	3090 实测（205）	4090 估算	说明
单卡渲染：稳定说话路数	3（有待机会话时）	4~5	算力约 2 倍，QuickTalk 不是纯算力瓶颈，保守按 1.5 倍
单卡渲染：在线上限 cap（1:2）	6	9	心跳按实测再调
单卡合成（VoxCPM2）	够 ~60 路在线	~100 路在线	合成只在说话时占用
单机 7 渲染 + 1 合成	—	**约 60 在线 / 30 同时说话**	渲染是瓶颈，合成略有余量
两台合计	—	**约 120 在线 / 60 同时说话**	单台宕机降到一半
200 在线目标	—	4 台（23 渲染卡 + 3 合成卡 + 预留）	与之前估算一致

若实测 4090 单卡能稳 5 路说话，cap 可调到 10，两台到 140 在线。

9.3 测试阶段要测出来、正式机才好配的数字

1. **每路会话的 CPU 占用**：WebRTC 推流/编码在 CPU 上且受 GIL 限制，63 路/机时 CPU 可能先于 GPU 见顶 → 决定 CPU 规格。
2. **每路带宽**：约 1~1.5 Mbps/路，63 路 ≈ 100 Mbps/机，TURN 中继翻倍。
3. **节点网关单进程能扛多少路**（ASR + 回调转发 + 心跳），不够就 ASR 单独进程。
4. **素材分发 + 全节点预热耗时**，决定 `avatar.ready` 的预期时间。
5. **主备切换各段实际时长**，写进给后端的 SLA。

9.4 正式机硬件建议（每台）

- GPU：8×RTX 4090 24G
- CPU：≥ 32 物理核（视 9.3-1 实测上调），内存 ≥ 256G
- 磁盘：NVMe ≥ 2T（模型 + 形象素材本地全量 + 日志/录制）
- 网络：万兆内网；公网固定 IP，放通 UDP 端口段（WebRTC + coturn）
- 系统：驱动 ≥ 550、docker + nvidia-container-toolkit、NTP、keepalived

十、落地步骤与验证

落地步骤（先在 205 + 206 验证架构）

1. **拆容器**：`talk-control`、`talk-node`、`talk-tts`；`entrypoint.sh` 按 `ROLE` 起进程。
2. **节点网关**（基于现有 `gateway/app.py` 瘦身）：本地会话表落盘、`/internal/*` 反代 + token、心跳、ASR、按会话选 LLM provider、超时定时器、事件 spool。
3. **控制面**：`ot_pool.py` 改节点注册表 + 全局 `pick()`；`/v1/*`、`/rt/*` + HMAC ticket；`/internal/events` 落盘后回调；素材分发与对账；不持 VIP 时拒绝受理。
4. `gateway/tts.py`：合成池列表、本机优先；talk-tts 未知音色时向控制面拉参考 wav。
5. 任务落盘（`animate.py` 的 `JOBS` → `state/jobs/`）。
6. 主备：备用控制面 + keepalived VIP（nopreempt + 健康脚本）；证书同步；TURN 每台一个。
7. 可观测：`/metrics`、`X-Request-Id` 透传。
8. `scripts/deploy.sh` + `scripts/hosts/*.env` 按角色部署。

验证

- 两机起来后 `GET /v1/health` 全部实例在线；`smoke.py` 指向 VIP 跑通。
- `loadtest.py` 开 14 路以上，会话分到两台机、无卡顿；心跳上报的在线数与控制面一致。
- 一台上建的形象 / 克隆的音色，另一台上的会话能直接用。
- 通话中 `docker stop` 一台渲染节点：15 秒内摘掉，前端重连落到另一台。
- drain 一台节点后新会话不再分过去，老会话正常结束。
- 停掉主机：VIP 漂移，后端无感；备机上会话不断，主机上会话收到 `node_failure` 后重连成功；切换前签发的 ticket 仍可用。
- 模拟网络分区：只有持 VIP 的一方受理，另一方 503，无重复回调。
- 杀掉控制面再拉起：节点 spool 事件补发、`event_id` 不重复；建形象任务续跑。
- 推送素材时让节点离线 5 分钟，恢复后对账补齐。

十一、离线生产线：闲时视频生成（H3 本地）与多模态人设构建

两类新需求都不是实时通话，属于「离线任务」：吃 GPU 的（H3 生成）和吃 API/CPU 的（人设抽取）。统一做成**一个离线任务队列**（控制面持有，落盘），和实时通话抢资源时实时优先。

11.1 形象变体：H3 本地闲时生成（背景、服装、姿态、待机动作）

要生成什么：同一个人脸的多套「底板视频」——换背景、换服装、换姿态/手势、不同待机动作。每套底板 = 一个 `variant`，挂在形象下（`av_xxx → var_xxx`），QuickTalk 口型融合不变。

为什么本地跑：云端 H3 按条计费且排队不可控；本地 H3 已在 205 跑通（`talk/h3/`：ComfyUI + INT8 FL2VA + NVFP4 文本编码器 + 4 步 LoRA），单张 24G 卡 544×736、6.6 秒片段约 160 秒，显存占满 → **一个 H3 任务独占一张卡**，不能和渲染同卡。4090 机驱动 ≥550 可用 INT8 视频 VAE，预计快 30~40%。

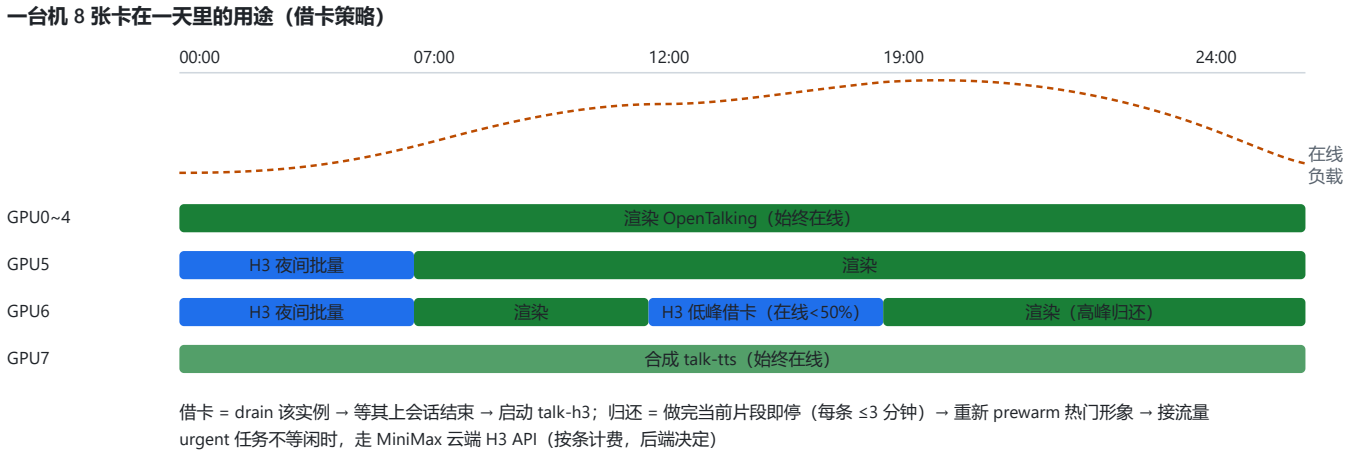


图 5 · H3 借卡策略：夜间每台借 2 卡、低峰借 1 卡、高峰不借

档位	条件	做法
夜间批量	00:00–07:00 或在线 < 20%	每台机 drain 2 个渲染实例，借 2 张卡跑 H3（`talk-h3` 容器按需启动，常驻会占显存）
白天低峰	在线 < 50% 持续 10 分钟	每台机借 1 张卡
高峰	在线 ≥ 50% 或任一实例满	不借卡；正在跑的任务做完当前片段即停、归还卡

- 借卡顺序固定（GPU6 → GPU5），归还相反，避免所有实例都被扰动。
- 产物流程：生成 → 首尾帧接缝检查（现有 `_seam_ratio`，不达标换种子重试 ≤3 次）→ 登记 variant → 推送所有节点 → 各节点 prewarm → 回调 `variant.ready`。
- 容量影响：夜间借 2 卡每台少 18 在线，白天借 1 卡少 9；两台一晚 ≈ **500 条底板**。

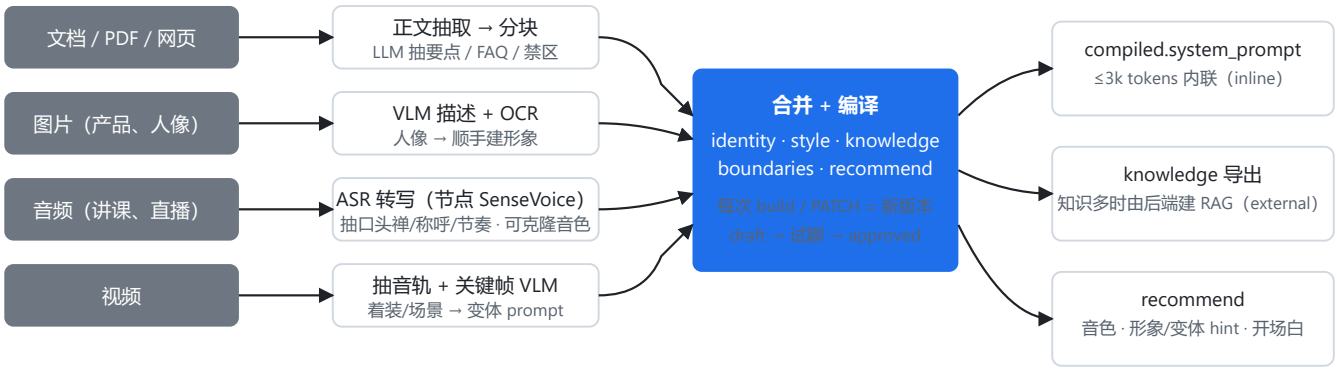
接口（后端 `/v1/*`）

方法	路径	说明
POST	`/v1/avatars/{id}/variants`	`{kind: background|outfit|pose|idle, prompt, reference_image?, duration_s: 6, seed?, priority: idle|normal|urgent, callback_url?, biz_ref?}` → `202 {variant_id, job_id, eta}`；`idle` 只在闲时跑、`normal` 低峰即跑、`urgent` 走云端
GET	`/v1/avatars/{id}/variants`	变体列表 `[{variant_id, kind, prompt, status: queued|generating|checking|distributing|ready|failed, preview_url}]`
GET	`/v1/variants/{vid}`	详情 + 预览视频 URL
DELETE	`/v1/variants/{vid}`	从所有节点删除
POST	`/v1/variants/{vid}/retry`	失败/不满意时换种子重做
GET	`/v1/offline/queue`	`{pending, running:[{job, node, gpu, progress}], borrowed_gpus, policy}`
PATCH	`/v1/offline/policy`	调三档阈值 / 夜间时段 / 每台最多借几张卡

会话里用变体：建会话加 `variant_id`（不传 = 默认底板）；通话中 `PATCH /v1/sessions/{id} {variant_id}` → 节点上已 prewarm 则在下一次待机间隙热切底板（需给 `opentalking_patch.py` 加「热切底板」，同一张脸的人脸缓存可复用）；不支持热切时兜底「同 sid 内部重建 OT 会话」，前端约 1 秒过渡，SSE 发 `session.variant_changed`。调度时优先挑已预热该 variant 的实例。

11.2 多模态素材 → 结构化人设

目标：客户上传素材（公司介绍 PDF、产品图、话术文档、讲课录音/视频、公众号链接……），AI 端抽取成**结构化人设**（`ps_xxx`），而不是让客户手写 system_prompt。人设可版本化、可审核、可复用到多个形象/音色。



会话：POST /v1/sessions {persona: {persona_id, version}}；knowledge_mode=external 的人设必须配 llm.mode=external

图 6 · 多模态素材到结构化人设的流水线（跑在控制面所在机，GPU 只用一点 ASR）

结构化人设 schema（v1）

```
{
  "persona_id": "ps_xxx", "version": 3, "status": "draft|approved",
  "identity":   {"name": "…", "role": "…", "org": "…", "intro": "一句话自我介绍"},
  "background": ["学历/经历/成就 要点"],
  "style":      {"tone": "亲切|专业|幽默…", "pace": "短句为主", "catchphrases": ["…"], "address
                 "dialect_hint": "四川话", "examples": [{"user": "…", "bot": "…"}]},
  "knowledge":  {"facts": ["…"], "faq": [{"q": "…", "a": "…"}], "products": [{"name": "…", "po
  "boundaries": {"forbidden": ["不谈竞品价格"], "fallback": "超出范围时怎么说", "handoff": "转
  "recommend":  {"voice_id": "vox_f_30s", "avatar_hint": "正装 办公室背景", "opening": "开场白
  "compiled":   {"system_prompt": "由以上字段编译", "prompt_tokens": 1800, "knowledge_mode": "
}
```

- 每份素材一个 `material`，可单独删除、重新抽取；人设 = 多份素材合并 + 人工修改，每次保存 +1 版本，会话固定引用某一版本。
- **知识量分档**：编译后 ≤3k tokens 直接内联（豆包 mini 首字 0.4 秒不受影响）；超过则只内联 identity/style/boundaries，知识走 `external` 由后端做 RAG。AI 端第一期**不做向量库。**
- 审核：`draft` 可文本试聊（不占 GPU），满意后 `approve`；只有 approved 版本能用于正式会话（后端可放开）。
- 存储：素材文件 + 人设 JSON 在控制面（落盘并同步备机）；归属/权限/计费在后端，用 `biz_ref` 关联。

接口（后端 `/v1/*`）

方法	路径	说明
POST	`/v1/personas`	`{name, biz_ref?, seed?}` → `{persona_id, version: 1}`；可从空白或手填字段起步
POST	`/v1/personas/{id}/materials`	multipart：`file` 或 `url`，`kind: doc|image|audio|video|url`，`hint?` → `202 {material_id, job_id}`；单文件 ≤200MB，音视频 ≤60 分钟
GET	`/v1/personas/{id}/materials`	素材列表 + 抽取状态/摘要
DELETE	`/v1/personas/{id}/materials/{mid}`	删素材并标记需重新合并
POST	`/v1/personas/{id}/build`	合并所有素材 → 新版本（`202 job`），回调 `persona.built{version, diff_summary}`
GET	`/v1/personas/{id}?version=`	结构化人设 JSON（默认最新版）
PATCH	`/v1/personas/{id}`	人工改任意字段 → 新版本并重新编译
POST	`/v1/personas/{id}/chat`	`{messages}` 文本试聊，流式返回，不占 GPU
POST	`/v1/personas/{id}/approve`	标记 approved；返回 `compiled` 摘要和 `knowledge_mode` 建议
GET	`/v1/personas/{id}/export`	导出 `compiled.system_prompt` + `knowledge` 全量（后端建 RAG 用）
DELETE	`/v1/personas/{id}`	删除人设及素材

会话里用人设：`POST /v1/sessions` 的 `persona` 支持 `{persona_id, version?, extra_prompt?}`（替代手写 system_prompt）；`knowledge_mode=external` 的人设必须配 `llm.mode=external`，否则 400。

回调事件新增：`material.extracted` / `material.failed`、`persona.built`、`variant.ready` / `variant.failed`。

11.3 离线任务队列（两类任务共用）

- 控制面维护 `state/jobs/`（落盘），任务带 `needs_gpu`、`priority`、`biz_ref`、重试次数。
- `needs_gpu` 任务只在借到卡时派发到对应节点的 `talk-h3` 容器（节点网关 `POST /internal/h3/run`，流式回进度）；非 GPU 任务在控制面本地线程池跑。
- 公平性：同一 `biz_ref` 最多同时 2 个任务在跑。
- 可观测：队列长度、等待时长、借卡时长、H3 每条耗时进指标；`/v1/offline/queue` 给后端做进度页。